

Relatório de Auditoria de Segurança

ControleRendaVariavel

1 de setembro de 2026

Escopo auditado	Repositório ControleRendaVariavel na íntegra: as 79 rotas HTTP dos 15 módulos de <code>`app/routes/`</code> , a fábrica da aplicação, as camadas de domínio e serviço, os modelos e as 13 revisões Alembic, os 40 templates Jinja, os 6 arquivos de JavaScript próprio, o gerenciador do coletor RTD (<code>`app/rtd_service.py`</code>), <code>`Dockerfile`</code> , <code>`compose.yaml`</code> , <code>`deploy/`</code> , os scripts PowerShell do agente Windows, os arquivos de ambiente (versionados e não versionados), o workflow de CI e o histórico completo do Git.
Linguagem	Python 3.14
Framework	Flask com blueprints por assunto (15 módulos de rota)
ORM / query builder	SQLAlchemy 2 (estilo <code>`select()`</code>) + Flask-SQLAlchemy; migrações por Alembic
Banco	PostgreSQL 17
Mecanismo de auth	SharedAuth v0.10 — sessão + Flask-Login, CSRF por Flask-WTF, portão padrão-nega, papéis <code>`admin`</code> / <code>`operador`</code> decididos localmente; mais um segundo mecanismo, por token Bearer, para o agente coletor
Frontend	HTML renderizado no servidor + HTMX (vendorizado) e Chart.js; fontes locais, sem CDN
Deploy	Docker Compose (gunicorn com 2 workers), Nginx com TLS no VPS Oracle, agente RTD rodando fora de contêiner numa máquina Windows
Superfície incomum	3 rotas <code>`/api/collector/*`</code> públicas por sessão e isentas de CSRF, autenticadas por token compartilhado com o agente Windows

Nota metodológica

Este aplicativo tem DOIS mecanismos de autenticação, e as cinco categorias foram aplicadas aos dois. O primeiro é o de pessoa (sessão + papel), que cobre 76 rotas. O segundo é o de máquina: três rotas ``/api/collector/*`` que ficam fora do portão de sessão e fora do CSRF, autenticadas por um token Bearer compartilhado com o agente que roda no Windows. Elas recebem atenção desproporcional neste relatório justamente por serem a única superfície do sistema alcançável sem sessão. Quanto ao isolamento por dono: o README declara que este não é serviço multiusuário, e a auditoria confirmou no schema — nenhuma tabela de domínio tem coluna de proprietário nas 13 revisões Alembic.

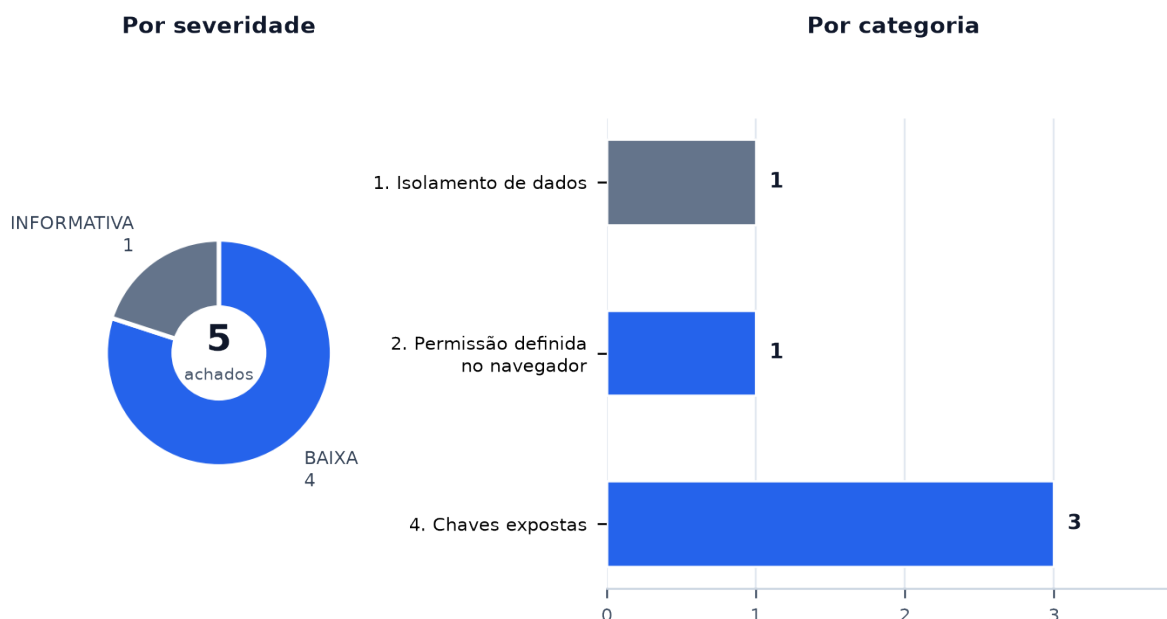
Categoria pedida	Como foi mapeada nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	Não há RLS nem middleware de tenant, e não há a que aplicá-los: posição, transação, provento e cotação pertencem a uma CARTEIRA, não a um usuário. O mecanismo efetivo é o portão padrão-nega mais o papel `admin` em 8 rotas. Auditadas todas as consultas de listagem, exposição, performance e risco.
2. Permissão definida no navegador	O único gate de papel do frontend (`base.html:173`) foi cruzado com as rotas do grupo que ele esconde, e as 79 rotas foram varridas para localizar `@requer_admin`.
3. IDOR	Percorridos os 24 handlers que recebem ID no caminho (`position_id`, `option_position_id`, `dividend_id`, `transaction_id`, `broker_id`, `ticker_id`, `portfolio_id`, `contract_id`, `expiration_id`, `user_id`). Como não há dono, a pergunta auditada foi: a rota confere PRIVILÉGIO quando é privilegiada, e valida a existência e a consistência do objeto?
4. Chaves expostas (hardcode)	Varredura de código, templates, `compose.yaml`, `Dockerfile`, scripts PowerShell (inclusive o de provisionamento do token do agente), workflow de CI, documentação, arquivos `.env` (inclusive os NÃO versionados) e do histórico completo do Git. Auditados os defaults `\${VAR:-valor}` do Compose e o `debug.log` presente no disco.
5. Inputs sem tratamento (XSS)	Frontend: varredura dos 6 arquivos de JavaScript próprio por `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval`, `new Function` e `document.write`. Backend: 40 templates buscados por ` safe`, `autoescape` e `Markup`; e atenção específica ao texto de erro que o agente coletor grava (`collector_agent_error`), que é dado de terceiro exibido na tela de Configurações.

1. Resumo executivo

O ControleRendaVariavel não tem nenhuma falha crítica, alta ou média. As 79 rotas foram percorridas uma a uma: as 8 privilegiadas (5 de gestão de contas, 2 de Configurações e o controle do serviço RTD) exigem `@requer\_admin` no servidor, e o único menu escondido por papel no template cobre exatamente esse conjunto. A varredura de XSS voltou vazia — zero `|safe` nos 40 templates, zero `innerHTML` nos 6 arquivos de JavaScript próprio, e o único `new Function` mencionado no código está num comentário explicando por que a CSP o impede. As duas superfícies mais expostas estão bem construídas: a API do agente coletor confere o token com `hmac.compare\_digest`, recusa com 503 quando o token não está configurado (falha fechada) e limita o corpo a 512 KB e 2.000 cotações; e a importação de histórico do Yahoo escapa o símbolo com `quote(symbol, safe='')` contra um host fixo, sem espaço para SSRF. Os cinco achados são de severidade baixa ou informativa: folgas de configuração, ausência de teto de taxa na única rota sem sessão, e decisões de projeto que valem estar registradas.



Achados por severidade e por categoria



2. Pontos fortes e pontos fracos

2.1 O que está protegido (com evidência)

Controle verificado	Evidência no código
Toda rota privilegiada é verificada no servidor As 8 rotas privilegiadas carregam <code>@requer_admin</code> , que recusa com 403 por <code>sharedauth.access.requer_papel</code> . O decorator ainda grava <code>papel_exigido</code> na view, para que uma varredura da URLconf consiga distinguir 'protegida por papel' de 'embrulhada por qualquer decorator'.	app/authorization.py:35-45; app/routes/users.py:52,58,76,89,105; settings.py:148,216; partials.py:53
O único menu escondido por papel corresponde exatamente às rotas protegidas O grupo 'Sistema' do <code>base.html</code> (Configurações e Usuários) é o único gate de papel do frontend, e o comentário ao lado já registra que esconder é apresentação: um operador que digite <code>/settings</code> recebe 403 mesmo sem ver o menu.	app/templates/base.html:170-193
API do agente coletor com comparação em tempo constante e falha fechada <code>_require_agent_token</code> compara o cabeçalho inteiro com <code>hmac.compare_digest</code> , e recusa com 503 quando o token não está configurado — em vez de deixar passar por comparação com string vazia. As três rotas são as únicas alcançáveis sem sessão, e todas passam por ele antes de qualquer outra coisa.	app/routes/collector_agent.py:41-48, 92, 150, 269
Limites defensivos na carga do agente Corpo recusado acima de 512 KB (com base em <code>content_length</code> , antes de ler), no máximo 2.000 cotações por chamada, e o <code>error</code> de falha cortado em 250 caracteres antes de gravar. Toda a gravação roda numa transação, com <code>rollback</code> em qualquer entrada inválida.	app/routes/collector_agent.py:27, 81-87, 156-157, 277
Importação do Yahoo sem espaço para SSRF O host é fixo (<code>query1.finance.yahoo.com</code>), o esquema é HTTPS literal e o símbolo do ticker — que é dado de usuário, cadastrado em <code>/tables/tickers</code> — entra escapado por <code>quote(symbol, safe="")</code> como segmento de caminho. Não há como redirecionar a requisição para outro destino.	app/quote_history_import.py:87-91
Acesso padrão-nega, com o motivo de cada exceção escrito <code>PUBLIC_ENDPOINTS</code> tem 6 entradas, e cada uma tem justificativa no próprio código — inclusive <code>sharedauth.static</code> , com a explicação de que sem ela a mensagem de erro do login apareceria sem estilo. <code>sharedauth_ui.static</code> foi deliberadamente deixado FORA, porque só <code>base.html</code> o referencia.	app/__init__.py:52-69
Isenção de CSRF explícita, nomeada e mínima Apenas as duas rotas POST do agente são isentas, e a isenção é feita por instância (<code>csrf.exempt</code> , não singleton de módulo), depois do registro dos blueprints. A terceira rota do agente é GET e não precisa de isenção.	app/__init__.py:281-285 e sharedauth/csrf.py
Sessão amarrada à senha em vigor, com guarda contra cookie de formato antigo O <code>user_loader</code> separa <code>id:marca</code> , recusa formato inválido antes do <code>int()</code> (um cookie antigo vira sessão recusada, não erro 500) e compara a marca contra o hash atual — trocar a senha derruba quem entrou com a antiga.	app/__init__.py:97-128
Zero XSS: nenhum <code>safe</code>, nenhum <code>innerHTML</code> Varredura dos 40 templates por <code>safe</code> , <code>autoescape</code> e <code>Markup</code> : nada. Varredura dos 6 arquivos de JavaScript próprio: nenhum <code>innerHTML</code> , <code>outerHTML</code> , <code>insertAdjacentHTML</code> , <code>eval</code> , <code>new Function</code> ou <code>document.write</code> — a única menção a <code>new Function</code> é um comentário explicando por que a CSP o impede. O <code>collector_agent_error</code> , que é texto de terceiro, sai pelo autoescape do Jinja.	app/templates/ (40 arquivos) e app/static/*.js

Controle verificado	Evidência no código
Subprocessos do coletor RTD construídos como lista, sem shell Todas as chamadas de <code>`subprocess.run`/`Popen`</code> passam a linha de comando como lista, sem <code>`shell=True`</code> . O único valor variável que entra num comando é um PID convertido com <code>`int()`</code> . Não há caminho de injeção de comando.	<code>app/rtd_service.py:105-122, 306-320, 355-360, 396-404</code>
Redirecionamento pós-login e pós-privacidade com a mesma checagem Tanto o <code>`next`</code> do login quanto o da alternância de privacidade passam por <code>`sharedauth.access.url_proximo_seguro`</code> — a rota de privacidade seria, sem isso, o redirecionador aberto que o login deixou de ser.	<code>app/routes/privacy.py:129-138</code> e <code>app/routes/auth.py</code>
Regra do último administrador com trava serializada <code>`update_user`</code> , <code>`create_user`</code> e <code>`upsert_from_cli`</code> tomam um lock antes de recontar administradores ativos, e recusam rebaixar o último. Cobre também o caso de reativar/desativar.	<code>app/user_management.py:65-72, 109-110, 151-154, 177-180</code>
Senha temporária nunca passa por <code>`flash()`</code> A senha gerada na redefinição vai como variável de contexto do template, e o comentário explica por quê: <code>`flash`</code> guarda na sessão, a sessão é um cookie ASSINADO e assinado não é cifrado — a senha sairia legível no cabeçalho da resposta.	<code>app/routes/users.py:37-48</code>
Sem segredo no histórico do Git Nenhum arquivo <code>`.env`</code> , <code>`.pem`</code> ou de segredo jamais foi adicionado ao repositório. Os dois scripts de provisionamento presentes no histórico geram segredo, não o contêm. O <code>`debug.log`</code> no disco foi lido por inteiro: não tem credencial nenhuma.	<code>git log --all --diff-filter=A</code>

2.2 Os riscos centrais

- O ``.env`` reúne num só arquivo, em texto claro, três segredos que também existem em ``.secrets/``: ``.SECRET_KEY``, ``.DATABASE_URL`` (senha incluída) e ``.POSTGRES_PASSWORD``. Não houve exposição — o arquivo já estava em ``.dropboxignore`` e nunca saiu da máquina —, mas aqui ele não é sobra inerte: ``.app/remote_collector_agent.py`` o abre diretamente para montar o ambiente do agente RTD.
- Não existe limite de taxa global: ``.iniciar_limiter`` é chamado sem ``.limites_padrao``, e só o login e dois fragmentos de polling têm teto. Isso deixa sem limite algum as três rotas ``.api/collector/*`` — as únicas alcançáveis SEM sessão em todo o sistema.
- As proteções de transporte moram fora da aplicação e o padrão do Compose é o inseguro (``.FORCE_HTTPS`` e ``.TRUST_PROXY_HEADERS`` em ``.false``). Agrava-se por algo específico deste projeto: ``.deploy/nginx/`` existe e está VAZIO — diferente do MegaSena e do ControleBancario, a configuração do proxy de produção não está versionada em lugar nenhum, e com ela vai embora o único ponto onde o HSTS existe.
- O acervo é comum por decisão de projeto: qualquer conta autenticada cria, edita e apaga posição, transação, provento e cotação de todas as carteiras. Está declarado no README e é coerente com um sistema de uso pessoal — mas o papel ``.operador`` existe, sugere separação, e não separa nada além de Configurações e Usuários.

2.3 Categorias que não se aplicam a esta stack

- **1. Banco sem tranca (isolamento de inquilino/dono)** — Não há inquilino nem dono a isolar. O README declara, na terceira linha, que a aplicação é 'de uso pessoal do mantenedor' e 'não é plataforma de negociação, custódia ou serviço multiusuário'. Confirmado no schema: posição, transação, provento e cotação apontam para CARTEIRA (``.portfolio_id``), e carteira não tem dono — nenhuma das 13 revisões Alembic introduz coluna de proprietário. Não há filtro de dono ausente; não existe filtro de dono a ter.

- **3. IDOR (objeto de outro dono)** — Pela mesma razão. Os 24 handlers que recebem ID foram percorridos assim mesmo, e todos resolvem o objeto com verificação de existência antes de agir; os 5 de ``/users/<id>/...`` exigem ``admin``. Não foi encontrado handler que altere objeto sem essa resolução.
- **5. XSS no backend (e-mail/template)** — A aplicação não envia e-mail e não monta HTML em Python. Toda saída passa por ``render_template`` com o autoescape do Jinja ligado, inclusive o texto de erro gravado pelo agente coletor.

3. Achados detalhados

Um bloco por categoria. Todo achado abaixo foi conferido no código real: arquivo, linha e trecho vêm da árvore auditada, não de inferência.

1. Isolamento de dados

Severidade	Arquivo:linha	Descrição
INFORMATIVA	app/routes/ (47 rotas de escrita) + README.md:3-6	[CRV-04] O papel `operador` sugere separação de dados e não separa nada Das 79 rotas, 47 gravam dado financeiro — posição, transação, provento, cotação, contrato de opção, corretora, carteira. Nenhuma delas exige papel. Só 8 rotas exigem `admin`: as 5 de gestão de contas, as 2 de Configurações e o controle do serviço RTD. Uma conta `operador` cria, edita e apaga qualquer registro de qualquer carteira. Por que é explorável: É a decisão de projeto funcionando como escrita — o README declara que a aplicação não é serviço multiusuário, e o schema confirma que não há dono. Registrado porque a existência dos DOIS papéis cria uma expectativa que o sistema não cumpre: quem vê `operador` na tela de Usuários tende a ler 'acesso reduzido aos dados', e o que existe é apenas 'não administra o sistema'. Impacto: Nenhum hoje, no uso pessoal do mantenedor. O ponto é que a distância entre o que o papel sugere e o que ele faz é exatamente o tipo de coisa que se descobre tarde, quando uma segunda pessoa recebe uma conta `operador` acreditando que ela é restrita. Condição de explorabilidade: Todas as 47 rotas exigem sessão. A trilha de auditoria (`app/auditoria.py`) registra as escritas financeiras por evento, então a ação fica rastreada mesmo sem ser impedida. <pre>@bp.post("/positions/<int:position_id>/delete") def delete_position(position_id: int): ... @bp.post("/transactions/<int:transaction_id>/delete") def delete_transaction(transaction_id: int):</pre>

2. Permissão definida no navegador

Severidade	Arquivo:linha	Descrição
BAIXA	app/__init__.py:213, 274-285 + app/routes/collector_agent.py:41-48	[CRV-02] Sem limite de taxa global: a API do agente, única superfície sem sessão, não tem teto nenhum `iniciar_limiter(app)` é chamado sem `limites_padrao`, então não há `default_limits`. Os únicos limites do sistema são `10 per minute` no login e `120 per minute` nos dois fragmentos de polling. As três rotas `/api/collector/*` — que estão em `PUBLIC_ENDPOINTS` e portanto ficam FORA do portão de sessão, e das quais duas são isentas de CSRF — não têm limite algum. Por que é explorável: É a única superfície do sistema alcançável sem sessão, e é a única sem qualquer contador. O token em si é longo e comparado em tempo constante, então adivinhá-lo não é uma via prática — mas a ausência de teto significa que qualquer origem na internet pode martelar `GET /api/collector/configuration` indefinidamente, e cada requisição consome um worker do gunicorn (são 2) e uma consulta ao banco antes de o 401 sair. Um 401 sem custo controlado é um alvo de negação de serviço. Impacto: Indisponibilidade do serviço a partir de uma origem NÃO autenticada — o que a distingue do mesmo achado nos irmãos, onde a rota cara exige sessão. Não há vazamento nem escalonamento. Condição de explorabilidade: Depende de a porta estar alcançável. O Compose publica apenas em `127.0.0.1:5301`, então a exposição real é a

Severidade	Arquivo:linha	Descrição
		do Nginx do VPS, que precisa alcançar essas rotas para o agente Windows funcionar (`REMOTE_COLLECTOR_ENABLED=true` no ambiente de produção). <pre>limiter = iniciar_limiter(app) ... aplicar_limite(app, limiter, "auth.login", LIMITE_LOGIN_PADRAO) aplicar_limite(app, limiter, ("portfolio.collector_heartbeat_partial", "portfolio.rtd_service_partial"), "120 per minute")</pre>

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
BAIXA	.env:1,3,10 (não versionado) + app/remote_collector_agent.py:81 + app/__init__.py:137-152	[CRV-01] Três segredos em texto claro num só `.env` — e o agente RTD o lê diretamente O arquivo `.env` guarda `SECRET_KEY=<redacted>`, `DATABASE_URL=<redacted>` e `POSTGRES_PASSWORD=<redacted>` em texto claro — os mesmos valores que já estão em `.secrets/secret_key` e `.secrets/postgres_password`, confirmado por comparação de hash. Por que é explorável: Não é exposição, e vale registrar o que foi verificado: o `.gitignore` recusa o arquivo (confirmado por `git check-ignore`); o histórico completo do Git não tem nenhum arquivo de segredo; o `.dropboxignore` já excluía `.env` da sincronização de nuvem ANTES desta auditoria (conferido em `git show HEAD:.dropboxignore`); e a pasta `.secrets/` foi verificada em dropbox.com, inclusive na lixeira — nunca subiu. O que resta é específico deste projeto e não some com a verificação: os TRÊS segredos estão no MESMO arquivo, e ele não é sobra de um caminho antigo — `app/remote_collector_agent.py:81` abre `.env` explicitamente para montar o ambiente do agente RTD, que roda no host Windows, fora de contêiner. Ou seja: o arquivo é operacional, e concentra num só lugar a chave que assina as sessões e a credencial do banco. Impacto: Nenhum comprometimento conhecido. O custo é de concentração: qualquer caminho que alcance esse único arquivo — um compartilhamento de pasta, um backup mal escopado, um arquivo compactado do projeto — entrega de uma vez a forja de sessão e o acesso ao banco, sem precisar de mais nada. Condição de explorabilidade: O `compose.yaml` operacional NÃO consome essas variáveis: concede os três segredos por arquivo Docker secret (`SECRET_KEY_FILE`, `POSTGRES_PASSWORD_FILE`, `COLLECTOR_AGENT_TOKEN_FILE`, linhas 73-75). O `.env` vale para a execução local e para o agente RTD no Windows. <pre>SECRET_KEY=<redacted></pre>
Severidade	Arquivo:linha	Descrição
BAIXA	compose.yaml:82-83 + app/__init__.py:137,193,202-207 + deploy/nginx/ (vazio)	[CRV-03] Padrões inseguros de transporte no Compose, e a configuração do proxy não está versionada `FORCE_HTTPS` e `TRUST_PROXY_HEADERS` valem `false` quando não informados. A primeira é o único interruptor que liga `Secure` nos cookies; a segunda decide se o `ProxyFix` é instalado. A aplicação não emite `Strict-Transport-Security` e não redireciona HTTP→HTTPS. E o diretório `deploy/nginx/` existe e está VAZIO — diferente do MegaSena (`deploy/nginx/megasena.conf`) e do ControleBancario (`deploy/nginx/controle-bancario.conf`), a configuração do proxy de produção deste projeto não está versionada em lugar nenhum.

Severidade	Arquivo:linha	Descrição
		Por que é explorável: As três garantias de transporte — cookie `Secure`, HSTS e chaveamento correto do limitador — dependem de duas variáveis de ambiente e de uma configuração de Nginx que existe apenas no servidor. Uma recriação do VPS não tem de onde restaurar a segunda: não há arquivo a copiar, só memória. E uma implantação que esqueça `.env.vps` sobe em silêncio com o cookie de sessão sem `Secure` e o limite de login chaveado pelo IP do gateway do Docker. Impacto: Cookie de sessão interceptável em requisição HTTP e proteção de força bruta do login sem efeito prático, numa implantação com as variáveis ausentes. A ausência do arquivo versionado transforma a perda do HSTS numa possibilidade real, e não hipotética, na próxima recriação do servidor. Condição de explorabilidade: A implantação atual está correta: `.env.vps.example` documenta `FORCE_HTTPS=true` e `TRUST_PROXY_HEADERS=true`. O achado é sobre a ausência de trava e sobre a configuração do proxy não estar sob controle de versão. <pre>FORCE_HTTPS: \${FORCE_HTTPS:-false} TRUST_PROXY_HEADERS: \${TRUST_PROXY_HEADERS:-false}</pre>
BAIXA	app/routes/collector_agent.py:41-48, 90, 148, 267	[CRV-05] Um único token de máquina, sem rotação prevista e sem escopo por operação As três rotas do agente compartilham um único `COLLECTOR_AGENT_TOKEN`: a de leitura da configuração, a que grava cotações e a que registra falha. O token é lido uma vez no boot e não tem prazo, contador de uso nem forma de revogação que não seja reiniciar o serviço; e ele vive em texto claro no host Windows, fora de contêiner, ao lado do agente que o usa. Por que é explorável: A verificação em si está correta (tempo constante, falha fechada). O que falta é o entorno: quem obtiver o token na máquina Windows — que não tem o endurecimento do contêiner: sem `read_only`, sem `cap_drop`, sem usuário dedicado — pode ler a configuração completa das posições abertas (`GET /api/collector/configuration` devolve ticker, código de mercado e lado de cada posição) e, sobretudo, GRAVAR cotações arbitrárias em `POST /api/collector/quotes`. Impacto: Escrever cotação falsa contamina preço corrente, histórico diário, cálculo de risco e desempenho — o produto inteiro do sistema. Não é vazamento de credencial de pessoa, é corrupção silenciosa de dado, difícil de perceber porque a origem é legítima. Condição de explorabilidade: Exige comprometimento da máquina Windows do agente ou do valor do token. O contador de páginas: `REMOTE_COLLECTOR_ENABLED` precisa estar ligado (é o caso em produção) para o agente existir. <pre>def _require_agent_token() -> None: configured = str(current_app.config["COLLECTOR_AGENT_TOKEN"]) supplied = request.headers.get("Authorization", "") if not configured: abort(503, "Coletor remoto nao configurado.") if not hmac.compare_digest(supplied, f"Bearer {configured}"): abort(401, "Nao autorizado.")</pre>

4. Recomendações priorizadas

Prio	Ação	Achados
P1	Aplicar limite de taxa às três rotas `/api/collector/*`. São as únicas alcançáveis sem sessão e as únicas sem contador. Um teto generoso o bastante para o agente legítimo e estreito o bastante para não deixar um 401 sem custo — por exemplo `60 per minute`, com `override_defaults=True`.	CRV-02
P2	Versionar a configuração do Nginx de produção em `deploy/nginx/`, que hoje é um diretório vazio, incluindo a diretiva de HSTS. Os dois irmãos que rodam no mesmo VPS já fazem isso; aqui a configuração existe apenas na memória do servidor, e some numa recriação.	CRV-03
P2	Fazer a aplicação recusar-se a subir quando `FORCE_HTTPS` estiver desligado num ambiente que declare proxy — a mesma trava recomendada aos irmãos, para que a configuração insegura deixe de ser silenciosa.	CRV-03
P3	Tirar os três segredos do `.env` e passar a leitura do agente RTD para `.secrets/`, via `app/secret_files.py`, que já existe e sabe fazer isso. Rotacionar não é necessário: os valores não vazaram.	CRV-01
P3	Separar o token do agente por operação, ou pelo menos separar leitura de escrita: um token que só lê a configuração e outro que grava cotação.	CRV-05
P3	Decidir e registrar o que o papel `operador` significa — restringir de fato as escritas destrutivas, ou renomeá-lo para algo que não sugira restrição de dados.	CRV-04

5. Issues para o GitHub

Cada bloco abaixo é o texto COMPLETO de uma issue, em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar ruído no rastreador.

Issue 1 — [Segurança] Três segredos concentrados num só `.env`, lido pelo agente RTD

```
--- ISSUE 1 ---
Título: [Segurança] Três segredos concentrados num só `.env`, lido pelo agente RTD
Labels: security, severity:baixa, segredos, higiene

## Problema

O arquivo `.env` guarda três segredos em texto claro, no mesmo lugar:

- `SECRET_KEY` — chave de assinatura da sessão;
- `DATABASE_URL` — URL de conexão, senha incluída;
- `POSTGRES_PASSWORD` — a mesma senha, de novo.

São os mesmos valores que já estão em `.secrets/secret_key` e
`.secrets/postgres_password`, confirmado por comparação de hash.

### O que NAO e

Não houve exposição. O que foi verificado:

- `.gitignore:34` recusa `.env`, confirmado por `git check-ignore`;
- o histórico completo do Git não tem nenhum arquivo de segredo;
- `.dropboxignore:33` já excluía `.env` da sincronizacao de nuvem ANTES desta auditoria — conferido em `git show HEAD:.dropboxignore`;
- a pasta `.secrets/` foi verificada em dropbox.com, inclusive na lixeira: nunca subiu.

### Por que ainda vale corrigir

Duas coisas que a verificação não desfaz:

1. Concentração. Os três segredos estão no MESMO arquivo. Qualquer caminho que alcance esse arquivo entrega de uma vez a forja de sessão e o acesso ao banco.
2. Ele é operacional, não sobra. `app/remote_collector_agent.py:81` abre `.env` diretamente para montar o ambiente do agente RTD, que roda no host Windows, fora de contêiner — sem `read_only`, sem `cap_drop`, sem usuário dedicado.

## Evidência

`app/remote_collector_agent.py:81`

```python
values = _read_dotenv(project_dir / ".env")
values.update(_read_dotenv(project_dir / CONFIG_PATH))
```

`compose.yaml:73-75` — o caminho que de fato vale em operação:

```yaml
SECRET_KEY_FILE: /run/secrets/secret_key
POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
COLLECTOR_AGENT_TOKEN_FILE: /run/secrets/collector_agent_token
```

## Impacto

Nenhum comprometimento conhecido. O custo é de concentração e de superfície: um arquivo que reúne tudo, num host que não tem o endurecimento do contêiner.

## Sugestão de correção

O projeto já tem o mecanismo certo e o usa em outro lugar:
`app/secret_files.py` lê segredo por arquivo do host, com recusa de ausente e vazio, e `.secrets/` já existe com quatro arquivos.

Mover a leitura do agente RTD de `.env` para `.secrets/`, via
`project_secret_value`, e deixar no `.env` apenas o que não é segredo
(`RTD_PROG_ID`, intervalos, flags). Rotacionar não é necessário: os valores não vazaram.

## Critérios de aceite

- [ ] `.env` não contém mais `SECRET_KEY`, `DATABASE_URL` nem `POSTGRES_PASSWORD`
- [ ] `app/remote_collector_agent.py` lê os segredos por `project_secret_value`, não de `.env`
- [ ] O agente RTD no Windows continua funcionando (verificado pelo pulso na tela de Configurações)
- [ ] A pilha local sobe com os segredos vindos de `.secrets/`
- [ ] `.dropboxignore` mantém `.env`, `.secrets/` e `.certs/` (regressão)
```

--- FIM ISSUE 1 ---

Issue 2 — [Segurança] API do agente coletor sem limite de taxa

--- ISSUE 2 ---

Título: [Segurança] API do agente coletor sem limite de taxa
Labels: security, severity:baixa, rate-limit

Problema

`iniciar\_limiter(app)` é chamado sem `limites\_padrao`, então não há `default\_limits`. Os únicos limites do sistema são `10 per minute` no login e `120 per minute` nos dois fragmentos de polling.

As três rotas `/api/collector/\*` não têm limite algum – e são justamente as que estão em `PUBLIC\_ENDPOINTS`, portanto **fora do portão de sessão**, e das quais duas são **isentas de CSRF**.

Por que é explorável

É a única superfície do sistema alcançável sem sessão, e a única sem qualquer contador. O token em si é longo e comparado em tempo constante, então adivinhá-lo não é uma via prática – mas a ausência de teto significa que qualquer origem pode martelar `GET /api/collector/configuration` indefinidamente, e cada requisição consome um worker do gunicorn (são 2) e uma consulta ao banco antes de o 401 sair.

Um 401 sem custo controlado é um alvo de negação de serviço.

Evidência

`app/\_\_init\_\_.py:213` e `274-280`

```
```python
limiter = iniciar_limiter(app) # sem limites_padrao
...
aplicar_limite(app, limiter, "auth.login", LIMITE_LOGIN_PADRAO)
aplicar_limite(
 app,
 limiter,
 ("portfolio.collector_heartbeat_partial", "portfolio.rtd_service_partial"),
 "120 per minute",
)
```
```

`app/\_\_init\_\_.py:54-59` – as rotas que ficam fora da sessão:

```
```python
PUBLIC_ENDPOINTS = frozenset({
 "auth.login",
 "portfolio.health",
 "portfolio.collector_agent_configuration",
 "portfolio.collector_agent_quotes",
 "portfolio.collector_agent_failure",
 "static",
})
```
```

Impacto

Indisponibilidade do serviço a partir de uma origem **não autenticada** – o que distingue este achado do mesmo problema nos aplicativos irmãos, onde a rota cara exige sessão. Não há vazamento nem escalonamento.

Sugestão de correção

Dois limites, ambos com `aplicar\_limite` (que já religa `view\_functions` corretamente):

```
```python
Generoso o bastante para o agente legítimo, que consulta a cada poucos
segundos; estreito o bastante para o 401 não sair de graça.
aplicar_limite(
 app, limiter,
 ("portfolio.collector_agent_configuration",
 "portfolio.collector_agent_quotes",
 "portfolio.collector_agent_failure"),
 "60 per minute; 2000 per hour",
 override_defaults=True,
)
```
```

E, opcionalmente, um `limites\_padrao` global para as demais rotas, no mesmo espírito do ConfortoTermico.

Critérios de aceite

- [] As três rotas `/api/collector/\*` têm limite dedicado

```
- [ ] O valor escolhido comporta o intervalo real do agente (conferir
      `agent_check_interval_seconds` em Configurações)
- [ ] Teste: chamada acima do limite responde 429, mesmo sem token válido
- [ ] Teste: o agente com o intervalo configurado não é bloqueado
- [ ] O pulso do coletor continua chegando na tela depois da mudança (verificado na tela)
--- FIM ISSUE 2 ---
```

Issue 3 — [Segurança] Configuração do Nginx de produção não está versionada e transporte depende de padrão inseguro

```
--- ISSUE 3 ---
Título: [Segurança] Configuração do Nginx de produção não está versionada e transporte depende
de padrão inseguro
Labels: security, severity:baixa, implantacao

## Problema

Duas coisas que se somam:

1. `FORCE_HTTPS` e `TRUST_PROXY_HEADERS` valem `false` quando não informados no
Compose. A primeira é o único interruptor que liga `Secure` nos cookies; a
segunda decide se o `ProxyFix` é instalado. A aplicação não emite
`Strict-Transport-Security` e não redireciona HTTP→HTTPS.
2. `deploy/nginx/` existe e está **vazio**. Diferente do MegaSena
(`deploy/nginx/megasena.conf`) e do ControleBancario
(`deploy/nginx/controle-bancario.conf`), a configuração do proxy de produção
deste projeto não está versionada em lugar nenhum – e é lá que o HSTS
moraria.

### Por que é explorável

As três garantias de transporte – cookie `Secure`, HSTS e chaveamento correto
do limitador – dependem de duas variáveis de ambiente e de uma configuração de
Nginx que existe **apenas no servidor**.

Uma recriação do VPS não tem de onde restaurar a segunda: não há arquivo a
copiar, só memória. E uma implantação que esqueça `.env.vps` sobe em silêncio,
com o cookie de sessão sem `Secure` e o limite de login chaveado pelo IP do
gateway do Docker – um balde único para todos os clientes.

## Evidência

`compose.yaml:82-83`

```yaml
FORCE_HTTPS: ${FORCE_HTTPS:-false}
TRUST_PROXY_HEADERS: ${TRUST_PROXY_HEADERS:-false}
...

`app/__init__.py:193` e `202-207`

```python
https_obrigatorio=bool(app.config["FORCE_HTTPS"]),
...
if app.config["TRUST_PROXY_HEADERS"]:
    app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1, x_host=1)
...

`deploy/nginx/` – diretório presente no repositório, sem nenhum arquivo.

## Impacto

Cookie de sessão interceptável em requisição HTTP e proteção de força bruta do
login sem efeito prático, numa implantação com as variáveis ausentes. A ausência
do arquivo versionado transforma a perda do HSTS numa possibilidade real – e não
hipotética – na próxima recriação do servidor.

## Sugestão de correção

1. Versionar a configuração do Nginx de produção em `deploy/nginx/`, incluindo
`add_header Strict-Transport-Security ... always;`, no mesmo formato que
`MegaSena/deploy/nginx/megasena.conf` já usa. Conferir com `sha256sum` dos
dois lados que o arquivo versionado é o que está no servidor;
2. acrescentar à fábrica uma checagem de boot que recuse `FORCE_HTTPS=false`
quando `TRUST_PROXY_HEADERS` estiver ligado (há proxy à frente = há TLS à
frente = o cookie precisa ser `Secure`).

## Critérios de aceite

- [ ] `deploy/nginx/` contém a configuração de produção, com HSTS
- [ ] O `sha256sum` do arquivo versionado bate com o do servidor
- [ ] Subir com `TRUST_PROXY_HEADERS=true` e `FORCE_HTTPS=false` levanta erro nomeando as duas
variáveis
- [ ] Subir sem proxy e sem HTTPS continua funcionando (desenvolvimento local)
- [ ] Teste cobrindo os dois casos
- [ ] `docs/deployment-vps.md` registra o arquivo novo e a checagem
```

--- FIM ISSUE 3 ---

Issue 4 — [Segurança] Token único do agente permite gravar cotação arbitrária

--- ISSUE 4 ---

Título: [Segurança] Token único do agente permite gravar cotação arbitrária
Labels: security, severity:baixa, coletor

Problema

As três rotas do agente compartilham um único `COLLECTOR\_AGENT\_TOKEN`: a que lê a configuração das posições abertas, a que grava cotações e a que registra falha. O token é lido uma vez no boot e não tem prazo, contador de uso nem forma de revogação que não seja reiniciar o serviço.

Ele vive em texto claro na máquina Windows do agente, ****fora de contêiner**** – sem `read\_only`, sem `cap\_drop`, sem usuário dedicado, que é o endurecimento de que os serviços do Compose desfrutam.

Por que é explorável

A verificação em si está correta e não é o problema: comparação em tempo constante, e falha fechada (503) quando o token não está configurado, em vez de comparar com string vazia.

O que falta é o entorno. Quem obtiver o token na máquina Windows:

- lê a configuração completa das posições abertas –
`GET /api/collector/configuration` devolve ticker, código de mercado e lado de cada posição, o que é a carteira inteira;
- e, sobretudo, ****grava cotações arbitrárias**** em `POST /api/collector/quotes`.

Evidência

`app/routes/collector\_agent.py:41-48`

```
```python
def _require_agent_token() -> None:
 configured = str(current_app.config["COLLECTOR_AGENT_TOKEN"])
 supplied = request.headers.get("Authorization", "")
 if not configured:
 abort(503, "Coletor remoto nao configurado.")
 if not hmac.compare_digest(supplied, f"Bearer {configured}"):
 abort(401, "Nao autorizado.")
```
```

O mesmo token guarda as três rotas: `app/routes/collector\_agent.py:92`, `150`, `269`.

Impacto

Escrever cotação falsa contamina preço corrente, histórico diário, cálculo de risco e desempenho – o produto inteiro do sistema. Não é vazamento de credencial de pessoa: é corrupção silenciosa de dado, e difícil de perceber porque a origem é legítima e a gravação é um `upsert` normal.

****Condição:**** exige comprometimento da máquina Windows do agente ou do valor do token, e `REMOTE\_COLLECTOR\_ENABLED` ligado (é o caso em produção).

Sugestão de correção

Separar leitura de escrita, que é a fronteira que importa aqui:

```
```python
Dois segredos, dois arquivos, dois provisionamentos independentes.
COLLECTOR_AGENT_TOKEN # le a configuracao (GET)
COLLECTOR_AGENT_WRITE_TOKEN # grava cotacao e falha (POST)
```
```

Assim o comprometimento da máquina Windows deixa de ser, automaticamente, autorização para corromper o histórico de preços – e a rotação do token de escrita pode ser mais frequente que a do de leitura.

Como defesa complementar, considerar rejeitar cotação cujo `observed\_at` esteja muito fora da janela do pregão, ou cujo preço divirja do último em mais de uma porcentagem configurável: um agente legítimo nunca produz esses valores.

Critérios de aceite

- [] Existem dois tokens distintos, um para leitura e outro para escrita
- [] `scripts/provision-collector-agent-token.ps1` provisiona os dois
- [] O agente Windows usa cada um na rota correspondente
- [] Teste: token de leitura recusado (401) em `POST /api/collector/quotes`
- [] Teste: token de escrita recusado (401) em `GET /api/collector/configuration`
- [] `docs/deployment-vps.md` registra a rotação independente dos dois

--- FIM ISSUE 4 ---

Issue 5 — [Segurança] Definir o que o papel `operador` restringe, ou renomeá-lo

```

--- ISSUE 5 ---
Título: [Segurança] Definir o que o papel `operador` restringe, ou renomeá-lo
Labels: security, severity:informativa, autorizacao, produto

## Problema

Das 79 rotas, 47 gravam dado financeiro – posição, transação, provento,
cotação, contrato de opção, corretora, carteira. **Nenhuma delas exige papel.**

Só 8 rotas exigem `admin`: as 5 de gestão de contas, as 2 de Configurações e o
controle do serviço RTD. Uma conta `operador` cria, edita e apaga qualquer
registro de qualquer carteira.

### Por que importa

Isso é a decisão de projeto funcionando como escrita – o README declara, na
terceira linha, que a aplicação é de uso pessoal e não é serviço multiusuário, e
o schema confirma que não há dono de dado. **Não é um defeito.**

O que gera risco é a distância entre o que o papel SUGERE e o que ele FAZ. Quem
vê `operador` na tela de Usuários tende a ler "acesso reduzido aos dados"; o que
existe é "não administra o sistema". É o tipo de coisa que se descobre tarde –
no dia em que alguém recebe uma conta `operador` acreditando que ela é restrita.

## Evidência

Rotas de escrita sem verificação de papel, exemplos entre 47:

```python
app/routes/positions.py:343
@bp.post("/positions/<int:position_id>/delete")
def delete_position(position_id: int):

app/routes/transactions.py:429
@bp.post("/transactions/<int:transaction_id>/delete")
def delete_transaction(transaction_id: int):

app/routes/dividends.py:235
@bp.post("/dividends/<int:dividend_id>/delete")
def delete_dividend(dividend_id: int):
```

Onde o papel É exigido – `app/routes/users.py:52`, `app/routes/settings.py:148`
e `216`, `app/routes/partials.py:53`.

`README.md:3-6`
```
Aplicacao web de uso pessoal do mantenedor para acompanhar acoes e opcoes: [...]
Nao e plataforma de negociacao, custodia ou servico multiusuario.
```

## Impacto

Nenhum hoje, no uso pessoal do mantenedor. A trilha de auditoria
(`app/auditoria.py`) registra as escritas financeiras por evento, então a ação
fica rastreada mesmo sem ser impedida.

## Sugestão de correção

Escolher uma das duas saídas coerentes e registrá-la:

**A – o papel passa a restringir de fato. As escritas destrutivas (exclusão de
posição, transação, provento e cotação) passam a exigir `admin`; criação e
edição continuam livres. É pouca mudança e alinha o papel com a expectativa.

**B – o papel deixa de sugerir o que não faz. Renomear `operador` para algo
que descreva o que ele realmente é ("usuário", "não administrador") e registrar
no README que o sistema não particiona dados por conta.

Seja qual for a escolha, ela merece um documento: é a decisão que faz duas das
cinco categorias desta auditoria não se aplicarem, e uma auditoria futura que não
a encontre registrada vai reabrir a discussão.

## Critérios de aceite

- [ ] A decisão (A ou B) está registrada em `docs/architecture.md` ou num ADR
- [ ] O documento nomeia o gatilho de revisão: a primeira conta de uma segunda pessoa
- [ ] Se A: as rotas de exclusão exigem `admin` e há teste por rota
- [ ] Se B: o papel foi renomeado na tela, no modelo e nas migrações necessárias
- [ ] `tests/test_authorization.py` reflete a decisão
--- FIM ISSUE 5 ---

```